OAuth Distributed Token Management - Deployment Guide

Overview

Objective: Prevent OAuth API rate-limit violations by sharing a single token across 8 application servers via database-backed coordination.

Key Metrics:

- OAuth API limit: 48 calls/day | Token validity: 1 hour | Token TTL: 45 minutes (2700 seconds) | Servers: 8 instances

---

Architecture Design

Two-Level Caching:

1. **L1 (In-Memory)**: Thread-safe volatile cached token per server
2. **L2 (Database)**: Single-row `OAUTH_TOKEN_CACHE` table shared across all servers

Concurrency Control:

- **Read**: Non-blocking SELECT (all servers read simultaneously)
- **Write**: Pessimistic locking with `LockMode.UPGRADE_NOWAIT` (single writer)
- **Conflict Resolution**: Failed lock attempts wait 500ms and retry read

Flow: Request → L1 Cache → L2 (Database) → Lock + Refresh → Wait/Retry

Guarantees: Only ONE server calls OAuth API at a time (DB lock enforced). MS SQL Server's `UPDLOCK, ROWLOCK, NOWAIT` prevents race conditions.

Database Setup

Execute DDL on e_Ngage Database

```
USE [e_Ngage];
GO

CREATE TABLE [dbo].[OAUTH_TOKEN_CACHE_TBL] (
    [ID] INT NOT NULL PRIMARY KEY DEFAULT 1
```

```
    [ID] INT NOT NULL PRIMARY KEY DEFAULT 1,
    [ACCESS_TOKEN] VARCHAR(4000) NOT NULL,
    [EXPIRY_TIME] BIGINT NOT NULL,
    [LAST_UPDATED] DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT CHK_OAUTH_SINGLE_ROW CHECK ([ID] = 1)
);
GO

-- CRITICAL: Initialize placeholder row
INSERT INTO [dbo].[OAUTH_TOKEN_CACHE_TBL] ([ID], [ACCESS_TOKEN], [EXPIRY_TIME], [LAST_UPDA
VALUES (1, '', 0, GETDATE());
GO

CREATE NONCLUSTERED INDEX [IDX_OAUTH_TOKEN_EXPIRY_NEW]
ON [dbo].[OAUTH_TOKEN_CACHE_TBL] ([EXPIRY_TIME]);
GO

-- Verify
SELECT * FROM [dbo].[OAUTH_TOKEN_CACHE_TBL];
GO
```

Expected: Single row with ID=1, empty token, EXPIRY_TIME=0

Application Deployment

Modified Files

1. **Entity**: `com.engage.hbean.OAuthTokenCache`
2. **DAO Interface**: `com.cmss.engage.dao.OAuthTokenCacheDao`
3. **DAO Implementation**: `com.cmss.engage.dao.OAuthTokenCacheDaoImpl`
4. **Service**: `com.cmss.engage.util.MiddlewareConnectionUtil`

Configuration

- **Auto-Discovery**: DAO auto-registered via `<context:component-scan base-package="com.cmss.engage" />`
- **Transactions**: Enabled via `<tx:annotation-driven />`, all DAO methods use `@Transactional`
- **Hibernate**: Add `<mapping class="com.engage.hbean.OAuthTokenCache"/>` if using hbm.xml

Pre-Deployment Checklist

- ☐ DDL executed on `e_Ngage` database
- ☐ Single row initialized (`SELECT COUNT(*) = 1`)
- ☐ Application user has SELECT/UPDATE permissions

- ☐ All 4 Java files compiled in WAR/EAR

Validation & Testing

Post-Deployment Verification

1. Database Check:

```
SELECT [ID], LEFT([ACCESS_TOKEN], 20) + '...' AS TOKEN,
       DATEADD(ms, [EXPIRY_TIME] % 1000, DATEADD(s, [EXPIRY_TIME] / 1000, '1970-01-01')) A
       [LAST_UPDATED]
FROM [dbo].[OAUTH_TOKEN_CACHE_TBL];
```

Expected: Non-empty token, expiry ~55 min future, recent LAST_UPDATED

2. Application Logs:

```
grep "OAuth" application.log | tail -20
```

Expected: ONE server logs `Lock acquired`, others log `Using token from database`. NO `All retry attempts exhausted`.

3. OAuth API Calls:

- **Before**: ~192 calls/day (8 servers × 24 refreshes)
- **After**: ~32 calls/day (1 refresh every 55 min)
- **Target**: Below 48 calls/day limit

Functional Tests

Test 1 - Token Refresh:

```
UPDATE OAUTH_TOKEN_CACHE_TBL SET EXPIRY_TIME = 0;
```

Make API call → Verify logs show refresh and database updated

Test 2 - Multi-Server Concurrency: Expire token, trigger simultaneous calls on 3+ servers → Verify exactly ONE acquires lock, others retry

Test 3 - Lock Timeout: Trigger concurrent requests → Verify servers log retry attempts with waits

---

Troubleshooting

Issue 1: Table Not Initialized

Symptom: `Token cache row not found (ID=1)` **Fix**: `INSERT INTO OAUTH_TOKEN_CACHE_TBL (ID, ACCESS_TOKEN, EXPIRY_TIME, LAST_UPDATED) VALUES (1, '', 0, GETDATE());`

Issue 2: All Retry Attempts Exhausted

Causes: DB connectivity issues | Long-running transactions | Clock skew **Fix**: Verify DB connection | Check for blocked sessions (`sp_who2`) | Sync server clocks (NTP)

Issue 3: Rate Limit Still Exceeded

Diagnosis: Check refresh frequency should be every 45-60 min **Possible Causes**: Incorrect TTL (`CON_EXP_OAUTH_TTL`) | Multiple databases | Token rejection **Fix**: Verify TTL=2700000 | Verify all servers use same DB | Check logs for 401/403 responses

Issue 4: Deadlocks

Cause: Multiple concurrent transactions (should NOT occur with single-row table) **Fix**: Verify single row exists | Review deadlock graphs: `DBCC TRACEON(1222, -1)`

Rollback Procedure

1. **Revert Code**: Deploy previous `MiddlewareConnectionUtil.java`
2. **Leave Database**: Do NOT drop table (safe for future redeployment)
3. **Verify**: Logs show local cache usage | OAuth calls increase to ~192/day | No `OAuthTokenCacheDao` errors

---

Monitoring & Operations

Key Metrics

1. **Token Refresh Rate**: ~32 refreshes/day | Alert if >48
2. **Lock Contention**: <10 `Could not acquire lock` /hour | Alert if >50/hour
3. **Token Expiry**: 0-45 minutes until expiry | Alert if negative

Manual Token Refresh (Emergency)

```
UPDATE OAUTH_TOKEN_CACHE_TBL SET EXPIRY_TIME = 0; -- Force refresh on next request
```

Performance

- **Fast Path** (L1 hit): <1 μs (99.9% of requests)
- **Slow Path** (L2 hit): 10-50 ms (0.1% of requests)
- **Refresh Path**: 200-500 ms (once per 45 min per cluster)

Configuration Properties

```
# OAuth API Configuration
CON_EXP_OAUTH_URL=https://oauth.provider.com/token
CON_EXP_OAUTH_TOKEN=Base64EncodedCredentials
CON_EXP_OAUTH_BODY=grant_type=client_credentials
CON_EXP_OAUTH_CONTENT_TYPE=application/x-www-form-urlencoded
CON_EXP_OAUTH_APIKEY=your-api-key
CON_EXP_OAUTH_TTL=2700000  # 45 minutes in milliseconds

# HTTP Client
CON_EXP_CONNECT_TIMEOUT=5000
CON_EXP_READ_TIMEOUT=10000
CON_EXP_IS_PROXY=false
DEFAULT_TIMEOUT_VALUE=30
```

Security Considerations

- **Token Storage**: Plain text in database (use TDE or column-level encryption if required)
- **Log Exposure**: Tokens NOT printed in logs (only prefixes)
- **Token Revocation**: API returns 401/403 → automatic refresh triggered

---

Document Version: 1.0 (Condensed) **Last Updated**: 2026-01-12 **Original Author**: Anvit Aigaokar